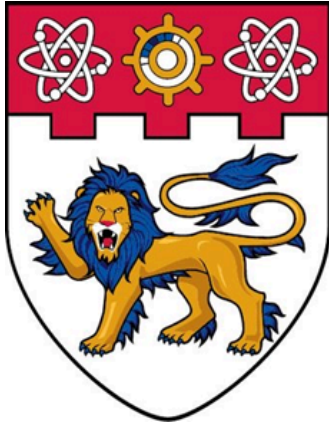




**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

SC2002 Object Oriented Design and Programming

Turn-Based Combat Arena

FCSJ | Group 1

Git Repository: <https://github.com/YongzhangLiu/sc2002-combat-arena>

Demo Video: <https://youtu.be/26Em07r7CUo>

Name	Matriculation Number
Kalepu Sai Sri Akshath	U2421041F
Tanusri Balasubramanian	U2421842F
Liu Yongzhang	U2223873E
Daniel Sutjipto	U2523054A
Abhijith Ravikumar	U2423981K

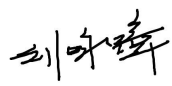
Declaration of Original Work

Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course (SC2002/CE2002 CZ2002)	Lab Group	Signature /Date
Kalepu Sai Sri Akshath	SC2002	FCSJ	 19/04/2026
Tanusri Balasubramanian	SC2002	FCSJ	 19/04/2026
Liu Yongzhang	SC2002	FCSJ	 19/04/2026
Daniel Sutjipto	SC2002	FCSJ	 19/04/2026
Abhijith Ravikumar	SC2002	FCSJ	 19/04/2026

Important notes:

1. Name must **EXACTLY MATCH** the one printed on your Matriculation Card.
2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

Declaration on Use of GAI Assistance

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Kalepu Sai Sri Akshath, saisriak001@e.ntu.edu.sg honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming
2. Course Code: SC2002
3. Instructor: Shen Zhiqi Li Fang
4. Title of Assignment/Project Submission: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

- i. Used GAI as permitted to assist in generating key ideas only. ☐
- ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

- iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

- iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ✓

I also declare that I have :

- a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
- b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
- c. Apart from the foregoing notices, the submission is wholly my own work.



Kalepu Sai Sri Akshath / 19 April 2026

Student Name & Signature Date

Declaration on Use of GAI Assistance

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Tanusri Balasubramanian, tanusri001@e.ntu.edu.sg honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming
2. Course Code: SC2002
3. Instructor: Shen Zhiqi Li Fang
4. Title of Assignment/Project Submission: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

- i. Used GAI as permitted to assist in generating key ideas only. ☐
- ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

- iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

- iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ✓

I also declare that I have :

- a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
- b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
- c. Apart from the foregoing notices, the submission is wholly my own work.



Tanusri Balasubramanian / 19 April 2026
Student Name & Signature Date

Declaration on Use of GAI Assistance

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Liu Yongzhang, M220173@e.ntu.edu.sg honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming
2. Course Code: SC2002
3. Instructor: Shen Zhiqi Li Fang
4. Title of Assignment/Project Submission: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

- i. Used GAI as permitted to assist in generating key ideas only. ☐
- ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

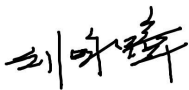
- iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

- iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ✓

I also declare that I have :

- a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
- b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
- c. Apart from the foregoing notices, the submission is wholly my own work.



Liu Yongzhang / 19 April 2026
Student Name & Signature Date

Declaration on Use of GAI Assistance

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Daniel Sutjipto, dani0064@e.ntu.edu.sg honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming
2. Course Code: SC2002
3. Instructor: Shen Zhiqi Li Fang
4. Title of Assignment/Project Submission: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

- i. Used GAI as permitted to assist in generating key ideas only. ☐
- ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

- iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

- iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ✓

I also declare that I have :

- a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
- b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
- c. Apart from the foregoing notices, the submission is wholly my own work.



Daniel Sutjipto / 19 April 2026
Student Name & Signature Date

Declaration on Use of GAI Assistance

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Abhijith, abhijith002@e.ntu.edu.sg honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming
2. Course Code: SC2002
3. Instructor: Shen Zhiqi Li Fang
4. Title of Assignment/Project Submission: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

- i. Used GAI as permitted to assist in generating key ideas only. ☐
- ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

- iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

- iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ✓

I also declare that I have :

- a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
- b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
- c. Apart from the foregoing notices, the submission is wholly my own work.



Abhijith Ravikumar / 19 April 2026
Student Name & Signature Date

Table of Contents

Declaration of Original Work.....	2
Declaration on Use of GAI Assistance.....	3
Introduction.....	9
UML Class Diagram.....	10
UML Sequence Diagram.....	18
Player's turn: Warrior defends against Goblin's attack on Warrior.....	18
Enemy's turn: Goblin executes basic attack, causing lower damage to Warrior.....	19
Additional Design Features.....	20
Custom Duel Mode.....	20
Quick Time Events.....	20
Graphical User Interface.....	20
Design Considerations.....	21
Inheritance and Polymorphism.....	21
Encapsulation.....	21
SOLID Design Principles.....	21
Trade-offs.....	22
Design Patterns.....	22
Summary Charts of Key Abstractions and Responsibilities.....	23
Reflection.....	26
Difficulties & Solutions.....	26
Knowledge Gained.....	26
Improvement Suggestions.....	27
Conclusion.....	27

Introduction

This turn-based combat arena game was developed in Java, applying Object-Oriented (OO) design goals and SOLID design principles. The game design was implemented with extensibility and maintainability in mind, demonstrating the practical application of the concepts learned in this course. Beyond the core requirements, we implemented a Lanterna-based Terminal User Interface (TUI), a Custom Duel Mode (CDM), and Quick Time Event (QTE) system inspired by video game design.

Our Github repository contains our Project ReadMe, Full Java Source Code, Maven Integration, Rendered UML Class and Sequence Diagrams (in SVG and all PlantUML diagram sources), Report, Team's Commit History (showing Individual Contribution) and a demo video.

UML Class Diagram

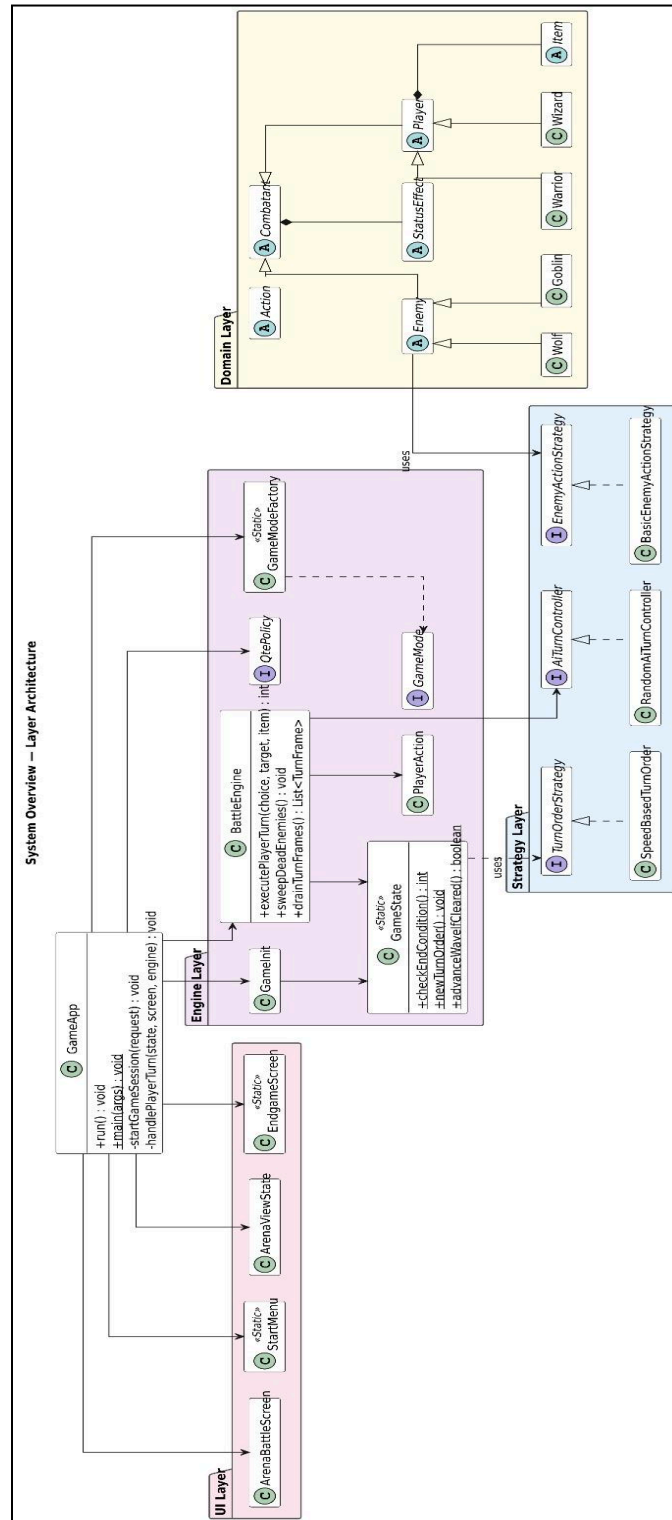


Figure 1: Overview of Full System Class Diagram in Landscape

The UML diagram is organised into 4 packages (*arena.ui*, *arena.engine*, *arena.strategy* and *arena.model*), with GameApp sitting outside any layer as the top-level orchestrator. The 4 design choices below explain briefly why the structure is shaped this way:

- **Layered architecture enforces Dependency Inversion at Package Level**
 - The UI package never imports engine logic and the engine logic never imports UI; both depend only on abstractions in strategy and model.
 - Communication from engine to UI happens through a read-only *ArenaViewState* DTO produced by *ArenaViewStateMapper*.
 - Replacing the Lanterna front-end with a CLI or web view requires no changes to combat logic, demonstrating DIP applied at architectural geometry.
- **All cross-cutting variability is expressed through interfaces**
 - *TurnOrderStrategy*, *EnemyActionStrategy*, *QtePolicy*, *GameMode* and *AiTurnController* are all polyphormic in nature.
 - *BattleEngine* calls them through their interfaces and never type-checks the concrete implementation.
 - Adding a new turn-order rule (e.g. an initiative roll), a new game mode (e.g. survival), or a new enemy AI requires only a new class, and no modifications to *BattleEngine* need to be done. This is the Open-Closed Principle made concrete in the diagram.
- **The Combatant hierarchy is the spine of the Liskov Substitution Principle**
 - Player and Enemy both extend *Combatant*.
 - The engine, the turn-order strategy and the status-effect hooks all operate on *Combatant*. This is what allows *SpeedBasedTurnOrder* to merge players and enemies into one sorted list, and what allows status effects like *StunEffect* to apply identically to a Goblin or to an 'AI-controlled' Warrior in Custom Duel mode.
- **Status effects and items are decoupled from combatants via Hook Pattern**
 - *StatusEffect* exposes hooks (on *TurnStart*, *modifyDefense*, *modifyIncomingDamage*) that *Combatant* calls at well-defined moments.
 - New effects (poison, regeneration, weakness) are added simply by subclassing *StatusEffect* and overriding the relevant hook.
 - The same shape applies to *Item* subclasses: each new item only needs to implement use (player, enemies, target).

The aforementioned notes are revisited in depth further into the report. Clear diagrams can be viewed directly on our **Github repository**.

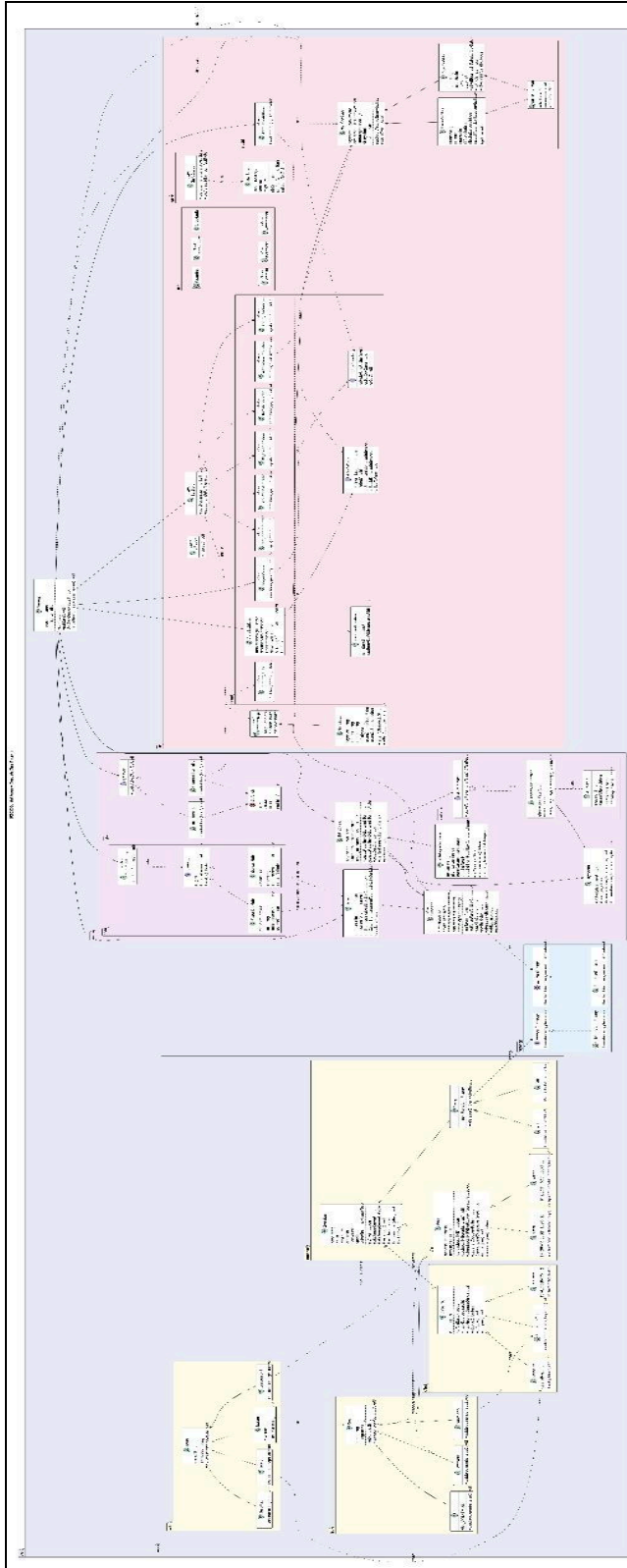


Figure 2: Full UML Class Diagram in Landscape

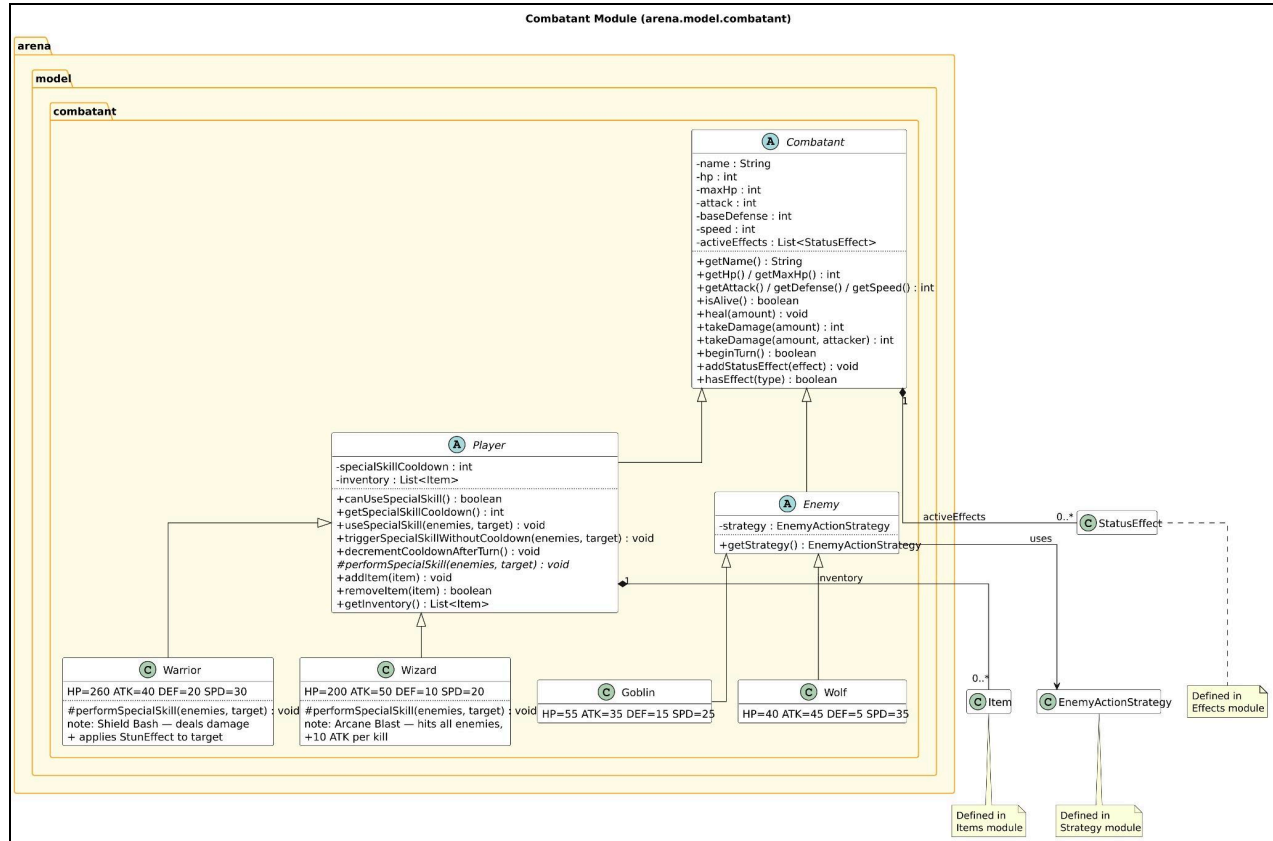


Figure 3: UML Class Diagram for Combatant Module

The Combatant Module models the combatant hierarchy. Combatant is the abstract base holding shared stats and behaviour (*takeDamage*, *beginTurn*), with *Player* and *Enemy* extending it, which is an application of inheritance and the Open-Closed Principle: new combatant types (e.g. new enemy) can be added without modifying existing classes. *Enemy* holds an *EnemyActionStrategy*, demonstrating composition over inheritance for behaviour variation.

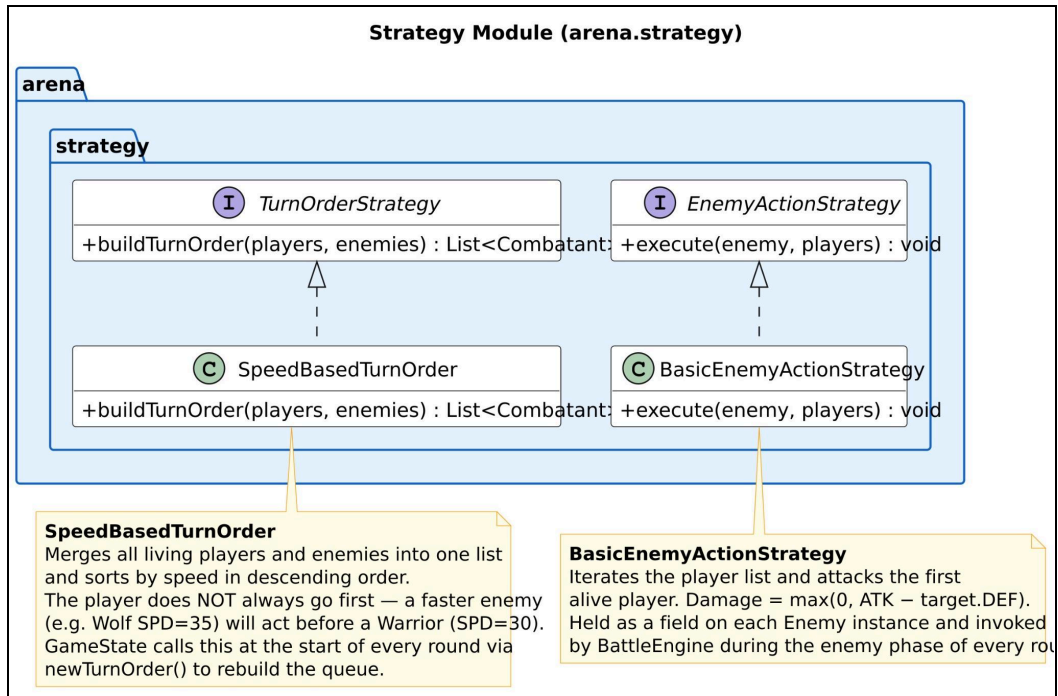


Figure 4: UML Class Diagram for Strategy Module

The module contains 2 interfaces (*TurnOrderStrategy* and *EnemyActionStrategy*), each with one concrete implementation.

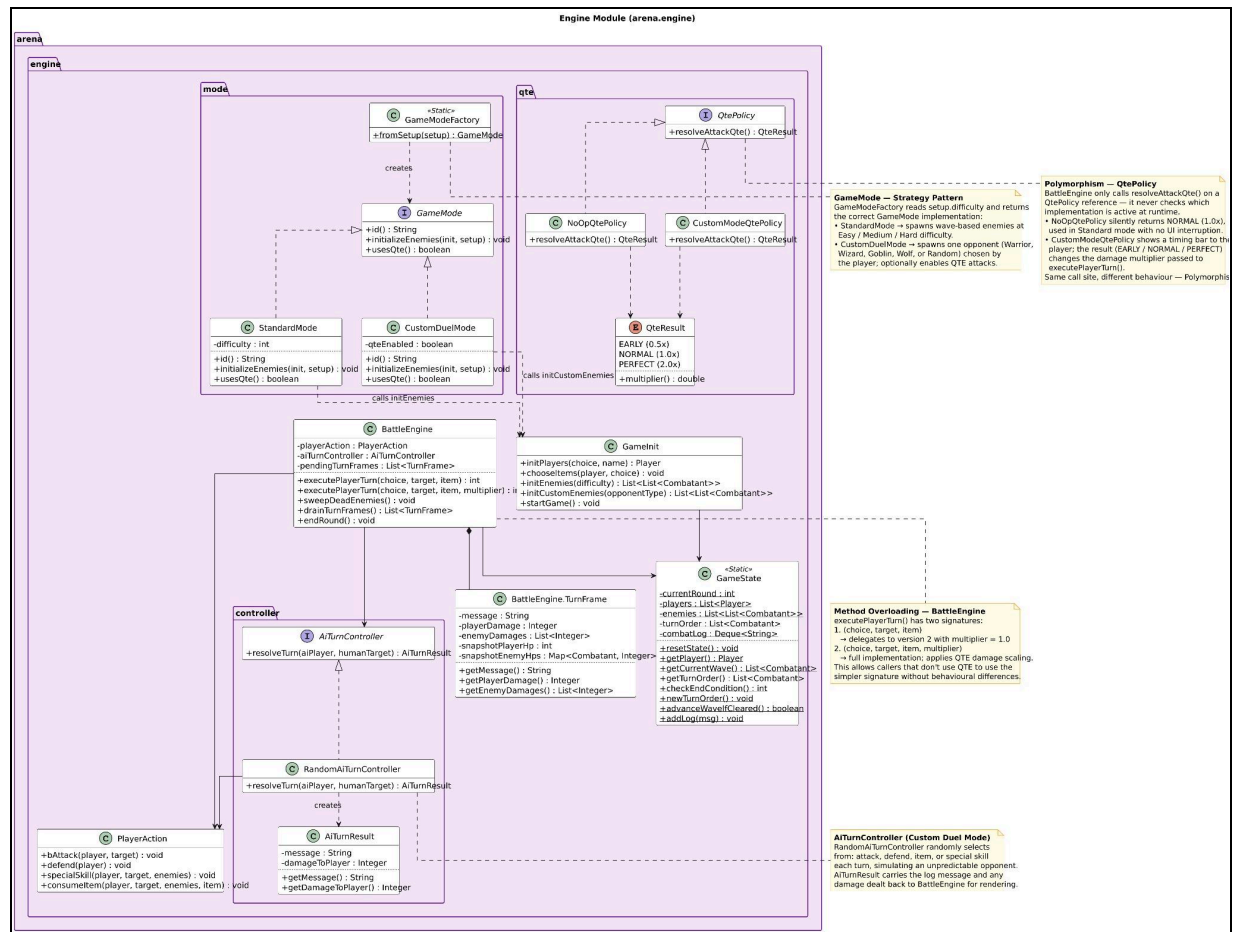


Figure 5: UML Class Diagram for Engine Module

The engine package manages game flow and state. *BattleEngine* drives each turn, delegating to *PlayerAction*, *GameMode*, and *GameState*. Notably, *GameModeFactory* uses the Factory Pattern to return the correct *GameMode* (Standard vs Custom Duel) based on setup (callers never instantiate modes directly).

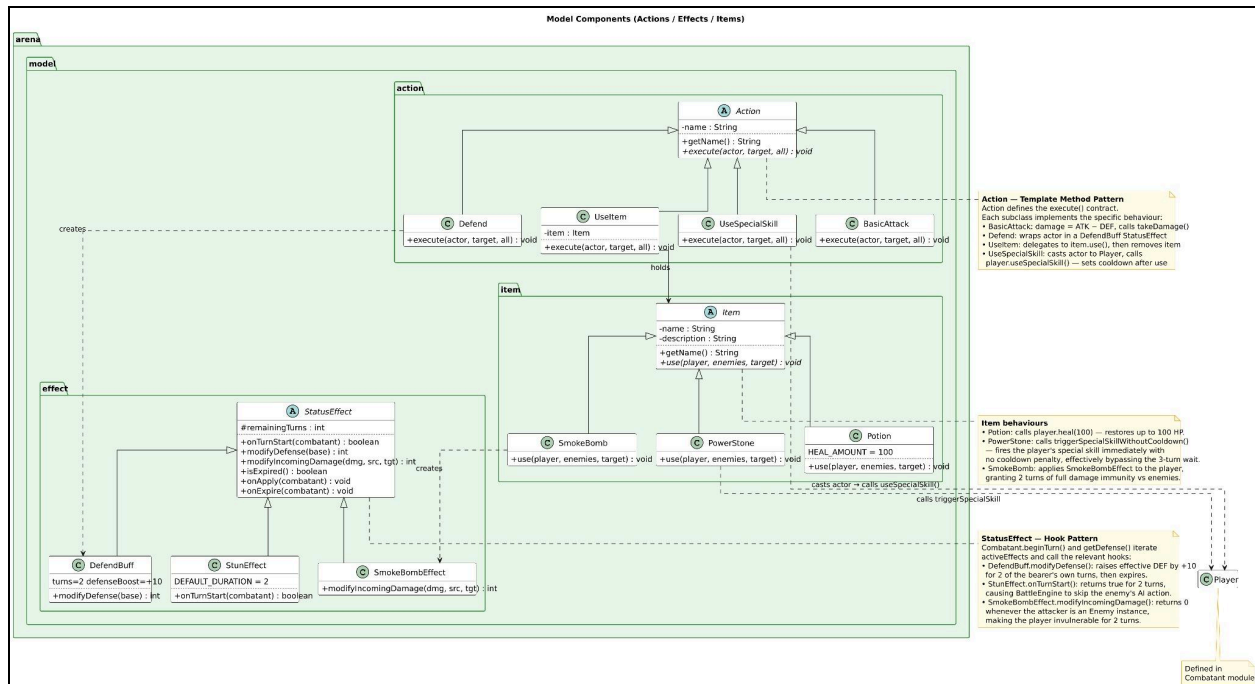


Figure 6: UML Class Diagram for Model Module

3 sub-packages implementing the Template Method Pattern (*Actions*) and Hook Pattern (*StatusEffects*):

- **Actions:** defines the `execute()` contract *BasicAttack*, *Defend*, *UseItem*, *UseSpecialSkill* each fill in specific behaviour
- **Items:** *Potion*, *PowerStone*, *SmokeBomb* extend *Item*, each with distinct `use()` effects (heal, bypass cooldown, damage immunity).
- **Effects:** *StatusEffect* exposes hook methods (`modifyDefense`, `modifyIncomingDamage`, `onTurnStart`) that subclasses override selectively, keeping *Combatant* clean of effect-specific logic.

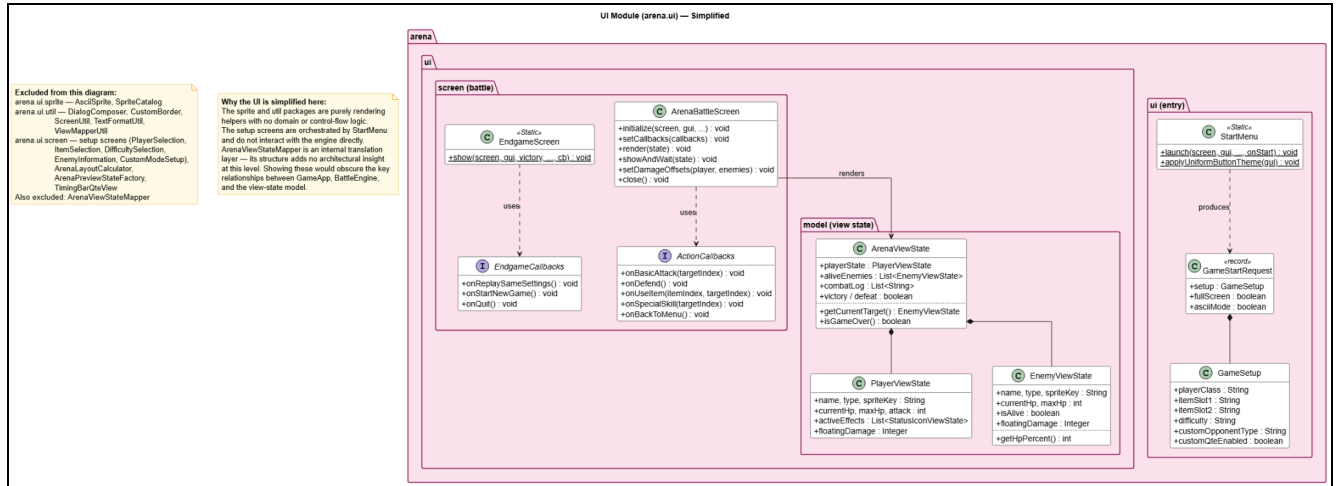


Figure 6: UML Class Diagram for UI

UML Sequence Diagram

Player's turn: Warrior defends against Goblin's attack on Warrior

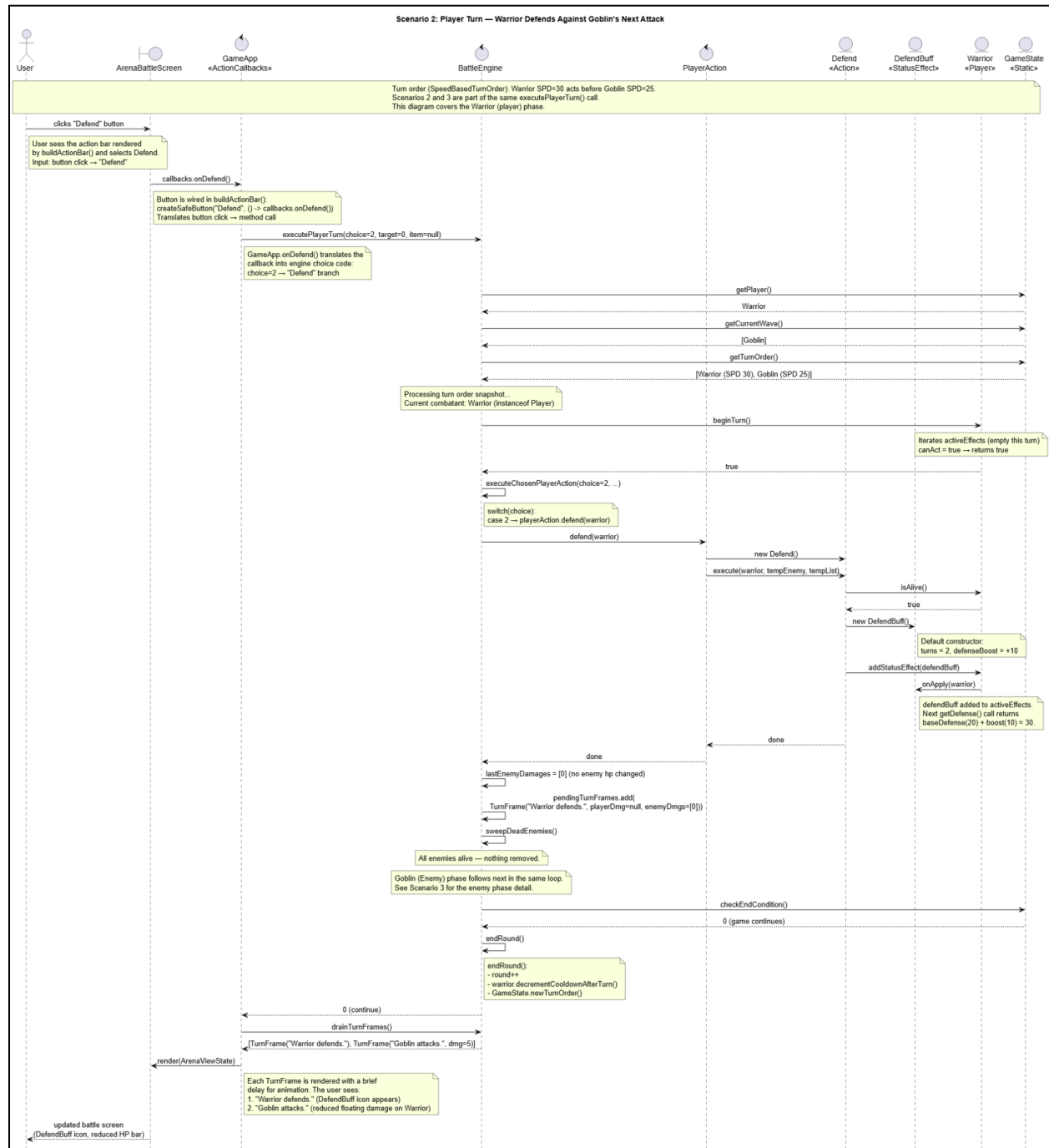


Figure 7: UML Sequence Diagram for Scenario 2

The user clicks Defend; *ActionCallbacks* routes the choice to *BattleEngine*; *PlayerAction.defend()* instantiates a *DefendBuff* and calls *warrior.addStatusEffect (buff)*. The buff is now an entry in the Warrior's *activeEffects* list, ready to influence subsequent damage calculations. The below sequence diagram depicts the consequent scenario.

Enemy's turn: Goblin executes basic attack, causing lower damage to Warrior

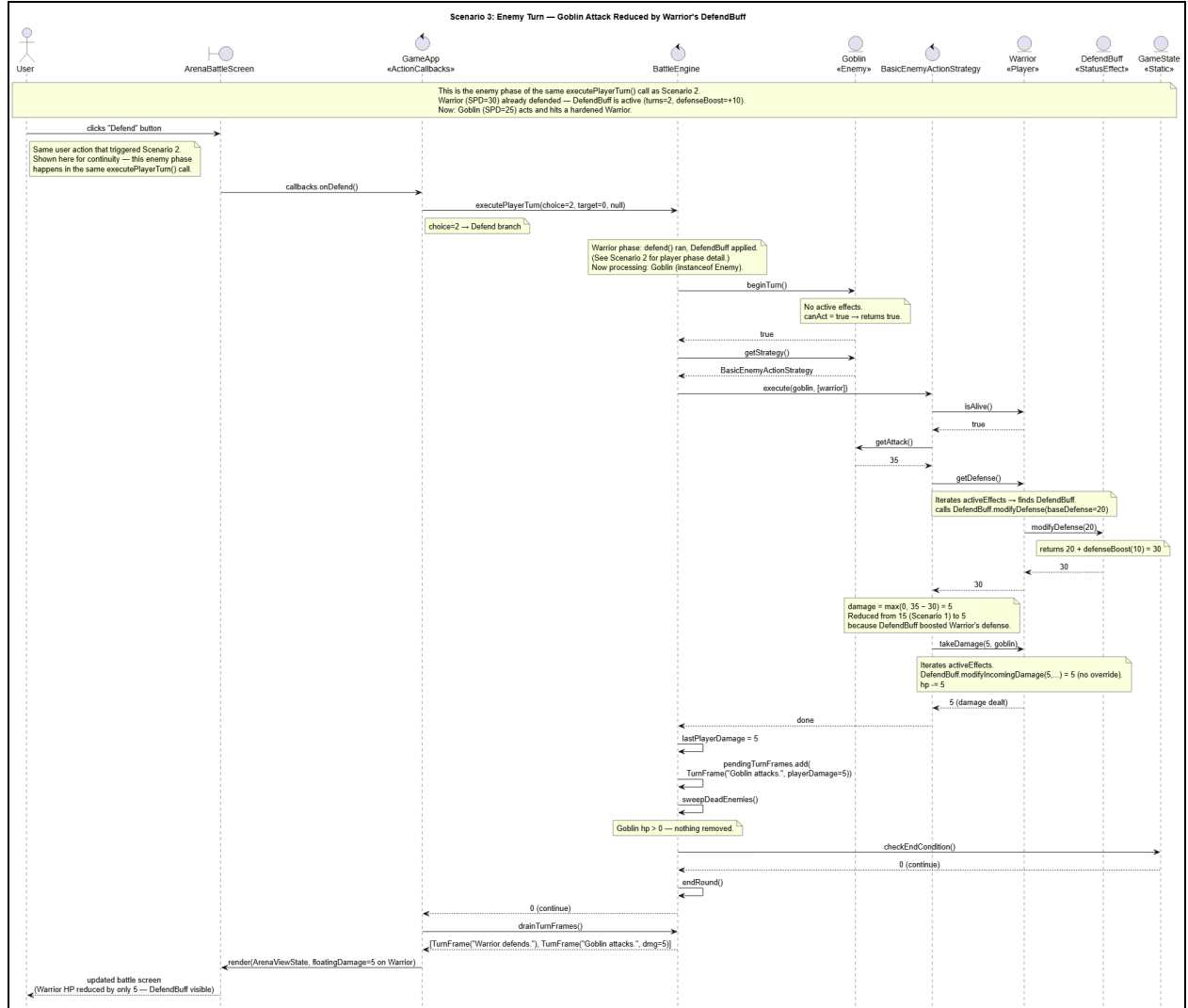


Figure 8: UML Sequence Diagram for Scenario 3

Additional Design Features

Beyond required core features, we have implemented several additional features that provide a unique and engaging game experience, while demonstrating OO design goals. This section covers our Custom Duel Mode, QTE system, and our choice of TUI.

Custom Duel Mode

The Custom Duel Mode allows the player to compete against a CPU-controlled combatant instead of the standard wave-based enemy encounter. The player can choose between a CPU-controlled combatant and a random combatant as their enemy. CPU-controlled combatants are initialised with randomly assigned items and make turn decisions through the *AiTurnController* interface, implemented by *RandomAiTurnController*. This encapsulates Artificial Intelligence (AI) behaviour behind an interface, meaning a more sophisticated decision-making strategy could be substituted without any changes made to the main engine. The *GameMode* interface allows the game to instantiate either *StandardMode* or *CustomDuelMode* based on the player selection, while *GameModeFactory* handles the decision. Hence, new game modes can be added via *GameMode* alone.

Quick Time Events

The QTE system is implemented to further support the Custom Duel Mode. When the player performs an attack, a timing bar appears and they must click at the right moment to achieve a damage multiplier.

- Too Early → press ‘z’ before end of timer → 0.5x multiplier (reduced damage)
- Too Late → press ‘z’ too late or misses → 1.0x multiplier (unchanged damage)
- Perfect → press ‘z’ exactly at the end of timer → 2.0x multiplier (doubled damage)

This is handled through the *QtePolicy* interface, with *NoOpQtePolicy* used for standard mode and *CustomModeQtePolicy* used for Custom Duel Mode. Then, the engine simply calls *resolveAttackQte()* and applies the returned multiplier. New QTE mechanics can thus be introduced via *QTEPolicy*.

Graphical User Interface

Instead of a Command-Line Interface (CLI), our game utilises a TUI using the Lanterna library, providing a more structured and visually engaging experience. The UI is completely separated from the game engine through a callback pattern in *ArenaBattleScreen*, meaning the engine does not depend on any UI implementation. A *TurnFrame* snapshot system was also implemented in *BattleEngine*, capturing the state after each action like HP changes and damage values. This decouples the UI animation logic from the battle resolution logic, allowing the UI to replay each action frame by frame with visual feedback.

Design Considerations

The following design considerations were applied throughout the development of the game to ensure correctness, extensibility, and maintainability. This section covers our application of inheritance, polymorphism, encapsulation and SOLID design principles. The trade-offs and design patterns considered are further elaborated as well.

Inheritance and Polymorphism

The combatant hierarchy is built from the shared abstract base class, Combatant, from which Player and Enemy extend, and are further specialised into Warrior, Wizard, Goblin, and Wolf. The abstract method performSpecialSkill() is defined in Player and overridden in each subclass, so that the engine can invoke the same method on any player without knowing the specific subtype. This demonstrates polymorphism and dynamic binding, eliminating the need for manual *instanceof* checks throughout the engine, making the codebase cleaner and easily extensible.

Encapsulation

All combatant states such as HP, attack, and defense are declared as private fields, accessible only through public getter methods and controlled mutators such as takeDamage() and heal(). This ensures that internal states cannot be modified from outside the class, supporting OO design goals of low coupling and high cohesion.

SOLID Design Principles

- **Single Responsibility Principle (SRP):** Each class was designed to fulfil one single responsibility. BattleEngine manages round execution, GameState holds shared game state, GameInit handles game initialisation, PlayerAction handles player action execution, and TurnOrderStrategy handles turn ordering. No single class was made to handle everything.
- **Open-Closed Principle (OCP):** The Action abstract class allows new actions to be added by creating new subclasses and implementing execute(), without modifying BattleEngine. Similarly, new StatusEffect types can be added without changing any existing combat logic and the QtePolicy and GameMode interfaces allow new QTE mechanics and modes to be introduced, respectively, without modifying existing code.
- **Liskov Substitution Principle (LSP):** Both Warrior and Wizard are substitutable as Player, and both Goblin and Wolf are substitutable as Enemy. The game engine operates on Combatant references throughout, and any subclass can be passed without breaking the system's behaviour.
- **Interface Segregation Principle (ISP):** Interfaces were kept small and focused. TurnOrderStrategy defines only buildTurnOrder(), EnemyActionStrategy defines only execute(), QtePolicy defines only resolveAttackQte(), and GameMode defines only methods required for mode initialisation. Hence, no class is forced to implement methods it does not use.

- **Dependency Injection Principle (DIP):** High-level modules such as *BattleEngine* and *GameApp* depend on abstractions rather than concrete implementations. *GameState* calls turn ordering through the *TurnOrderStrategy* interface, and *GameApp* depends on *OtePolicy* and *GameMode* interfaces. This ensures that high-level modules are not tightly coupled to low-level details, making the system easier to extend and maintain.

Trade-offs

GameState was implemented as a static class, providing convenient global access to game state across all engine components. However, this can introduce tight coupling between components and makes testing harder, since state cannot be easily reset between each test scenario. However, simplicity was prioritised given our project timeline, though refactoring it into an object could be a worthwhile improvement for larger systems.

The *TurnFrame* snapshot system adds memory overhead since each action creates a new frame object. However, separating UI animation from battle resolution was prioritised to ensure that the UI can be updated independently without affecting engine correctness.

Design Patterns

The Strategy Pattern was applied in multiple areas of the game design. This pattern defines a family of interchangeable algorithms behind a common interface, allowing behaviors to be swapped without modifying the classes that use them, directly supporting OCP from SOLID.

TurnOrderStrategy encapsulates the turn ordering algorithm. Particularly, *SpeedBasedTurnOrder* is the current concrete implementation, but a different ordering system can be introduced by implementing the same interface without any changes to *BattleEngine* or *GameState*. *EnemyActionStrategy* encapsulates enemy behaviour and is injected directly into *Goblin* and *Wolf* through *BasicEnemyActionStrategy*. Thus, an enemy AI, for example, can be introduced by swapping the strategy, again without the need for modifying the engine.

Thus, the Strategy Pattern was chosen over alternatives such as inheritance-based behavior. The latter avoids complex class hierarchies and allows behaviour to be swapped at runtime without subclassing. Thus, our turn-based combat arena game has a more flexible and maintainable design.

Summary Charts of Key Abstractions and Responsibilities

The following tables summarise key abstractions, their types, responsibilities and the relevant design principles applied.

Abstraction	Type	Responsibility	Design Principles
Combatant	Abstract Class	Shared base for all combatant participants. Owns HP, attack, defense, speed, status effects, and turn logic	Inheritance, Encapsulation, Polymorphism
Player	Abstract Class	Extends <u>Combatant</u> . Adds inventory, special skill cooldown, and abstract <u>performSpecialSkill()</u>	Polymorphism, OCP
Enemy	Abstract Class	Extends <u>Combatant</u> . Holds an <u>EnemyActionStrategy</u> for behaviour	Strategy Pattern, DIP
Warrior/Wizard	Concrete Class	Specialise <u>Player</u> with unique stats and <u>performSpecialSkill()</u> implementations	LSP, Polymorphism
Goblin/Wolf	Concrete Class	Specialise <u>Enemy</u> with unique stats and inject <u>BasicEnemyActionStrategy</u>	LSP, Strategy Pattern

Action	Abstract Class	Defines the <u>execute()</u> contract for all combat actions	OCP, SRP
BasicAttack, Defend, UseItem, UseSpecialSkill	Concrete Class	Each encapsulated one specific action behaviour	OCP, SRP
StatusEffect	Abstract Class	Defines hooks (<u>onApply</u> , <u>onTurnStart</u> , <u>modifyDefense</u> , <u>modifyIncomingDamage</u>) for persistent effects	OCP, Polymorphism

DefendBuff, StunEffect, SmokeBombEffect	Concrete Class	Each implements a specific effect with its own duration and modification logic	OCP
---	----------------	--	-----

Item	Abstract Class	Defines the <u>use()</u> contract for all usable items	OCP, SRP
Potion, PowerStone, SmokeBomb	Concrete Class	Each implements a single item behaviour	OCP, SRP

TurnOrderStrategy	Interface	Abstracts the algorithm for building turn order	OCP, DIP, Strategy Pattern
SpeedBasedTurnOrder	Concrete Class	Implements speed-based turn ordering	Strategy Pattern
EnemyActionStrategy	Interface	Abstracts enemy behaviour per turn	OCP, DIP, Strategy Pattern
BasicEnemyActionStrategy	Concrete Class	Implements basic attack behaviour for enemies	Strategy Pattern

GameMode	Interface	Abstracts game mode initialisation and configuration	OCP, DIP
StandardMode/CustomDuelMode	Concrete Classes	Define enemy setup and QTE usage per mode	OCP
GameModeFactory	Class	Decides with <u>GameMode</u> to instantiate based on setup	SRP

QtePolicy	Interface	Abstracts QTE resolution logic	OCP, DIP, ISP
NoOpQtePolicy/CustomModeQtePolicy	Concrete Classes	Provide mode-specific QTE behavior	OCP, Strategy Pattern
AiTurnController	Interface	Abstracts AI decision-making for CPU combatants	OCP, DIP
RandomAiTurnController	Concrete Class	Implements random action selection for CPU	Strategy Pattern

BattleEngine	Concrete Class	Does round execution, turn order processing, and action resolution	SRP
GameState	Static Class	Centralised storage for shared game state	SRP
GameInit	Concrete Class	Handles player creation, item assignment, and enemy wave setup	SRP, OCP
PlayerAction	Concrete Class	Bridges player input choices to <u>Action</u> executions	SRP

StartMenu	Concrete Class	Orchestrates UI session and screen navigation	SRP, DIP
ArenaBattleScreen	Concrete Class	Renders battle UI from view state	SRP, DIP
ActionCallbacks	Interface	Contract for user action handling	DIP, Polymorphism
ArenaViewState	Immutable Class	Snapshot of arena state for UI	Encapsulation, OCP
ArenaViewStateMapper	Concrete Class	Transforms domain to view state	SRP, OCP
ArenaLayoutCalculator	Concrete Class	Computes panel bounds from terminal size	SRP, Encapsulation

GameSetup	Mutable Class	Accumulates setup data across screens	Encapsulation, SRP
-----------	---------------	---------------------------------------	--------------------

Reflection

As a team, we had a comprehensive experience creating our turn-based combat arena game. We were able to apply concepts learnt from the course and learn from any challenges encountered to create solutions. This section covers the challenges we faced and how we mitigated them, knowledge gained, and improvement suggestions.

Difficulties & Solutions

Given the liberty to decide on the scope of the game, we were excited to implement many features simultaneously, from different item systems to UI elements. However, implementing too many features, especially at earlier stages of development, made the codebase and workflow messy and difficult to manage. A group meeting to align on scope and workflow was valuable in resolving this. As a team, we decided and completed the core gameplay loop first, before implementing additional features, adopting an incremental design approach. This mirrors common practice in professional software development, where a stable baseline must exist before further features are layered on top.

A key technical challenge was designing the class hierarchy for combatants. With multiple subclasses each requiring unique behaviours such as special skills, we initially struggled with how to handle differences without repeated *instanceof* checks. We resolved this by carefully designing shared parent classes with abstract methods, and relying on dynamic binding through method overriding to let each subclass define its own behaviour.

With regard to our TUI implementation, some features are not supported on the Windows terminal. While the game runs successfully in the native terminal of Linux and Darwin, Java still falls back to a *SwingTerminalFrame* for rendering. This results in nullified mouse clicking support and a smaller unicode character set. Hence, the visual experience can differ across systems. Similarly, since the Sprites are of set size, the screen real estate was not always sufficient for rendering. Hence, the minimum screen size of 120x42 pixels was set. Additionally, full screen view was set as default and all on-screen visual components are separated into designated locations. When multiple on-screen components are required, arrangement is dynamically calculated for uniform distribution.

Knowledge Gained

Working as a team, we learnt the importance of foundational planning. Establishing a clear framework before building features can prevent inefficiency. Thus, a core design allowed us to implement subsequent creative features more smoothly. As aforementioned, this is common practice in business environments, especially to ensure business objectives are aligned with technical design.

This project gave us a hands-on opportunity to view how OO design goals and SOLID principles work in practice. In particular, OCP and DIP were crucial for extensibility. For example, introducing Custom Duel Mode required no changes to BattleEngine at all. We also worked with interfaces such as TurnOrderStrategy, EnemyActionStrategy, QtePolicy, and GameMode. This practice reinforced how designing with regard to abstractions can make a system significantly easier to extend over time.

Improvement Suggestions

As aforementioned, instead of GameState being implemented as a static class, refactoring it into an injectable object can reduce hidden dependencies and improve testability.

When controlling turns, instead of random action selection, a more sophisticated decision tree can be implemented through the existing AiTurnController interface without any engine changes.

Additional game modes, difficulty levels or custom enemy compositions can be added by extending the GameMode interface. The QTE system can also be extended to other game modes by swapping the QtePolicy implementation.

These improvements can elevate the gaming experience for a larger more sophisticated game design.

Conclusion

Our turn-based combat arena game provided us with a meaningful opportunity to OO design in a familiar and engaging context. Through incremental development and application of SOLID principles and the Strategy Pattern, we built a system that is not only functional but also extensible and maintainable.